

Регулярные выражения

1. Основные понятия

Регулярное выражение — это строка символов, которая предназначена для поиска фрагментов текста в текстовой строке.

Регулярное выражение состоит из *шаблона* и *флагов*.

Шаблон — это строка символов, в которой описаны свойства искоемых фрагментов.

Флаги — это набор символов, которые задают параметры поиска.

В состав регулярного выражения флаги могут и не входить (параметры поиска задаются по умолчанию).

2. Обычные и специальные символы шаблона

Шаблон регулярного выражения может включать *обычные* и *специальные* символы.

Обычные символы — это буквы (русские и латинские), цифры, " " (пробел), "=" (знак равенства), "-" (дефис), "_" (знак подчеркивания), ",", "@" (знак "собака").

Специальные символы — это символы ".", "/" (дробная черта), "\" (обратная дробная черта), "*" (звездочка), "+" (плюс), "?" (знак вопроса), "(", ")", "[", "]", "{", "}" (скобки), "|" (вертикальная черта), "^" (степень), "\$" ("доллар").

Шаблон обычно описывается в форме литерала, в котором описание начинается и кончается символами дробной черты "/".

Например: /abc/, /co.e/, /ab?c/, /^(ab)+c/.

Определение. Шаблоны, составленные только из обычных символов, часто называют *простыми*.

Например, /abc/, /code/ — простые шаблоны.

Простой шаблон /code/ позволяет искать в строке символьный фрагмент "code". Например, с этим шаблоном будут найдены фрагменты "code" в строках

"In code we trust" и "I am coder".

Специальные символы, включенные в шаблон наравне с обычными, как обычные символы не воспринимаются.

Например, шаблон /co+e/ не имеет отношения к поиску фрагмента "co+e". Знак "+" как обычный символ не воспримется.

Специальные символы можно использовать как обычные, если перед ними поставить специальный символ "\". Например, шаблон /co\\+e/ уже позволяет искать фрагмент "co+e".

Это верно и для самого специального символа "\". Например, шаблон /co\\e/ позволяет искать фрагмент "co\е".

Каждая безошибочная часть шаблона называется *частичным шаблоном*. Частичный шаблон, заключенный в круглые скобки, называется *группой символов*.

3. Метасимволы

Метасимволы — это специальные символы или наборы символов, которые служат для сокращения записи или описания различных вариантов записи.

Основные метасимволы:

Метасимвол	Означает	Пример шаблона	Пример позволяет искать фрагменты
\	перед служебным символом переводит служебный символ в обычный	/abc\./ /a\\bc/	"abc." "a\bc"
.	обозначает любой единичный символ, кроме символа перевода строки	/abc./	"abca", "abcb", "abc1" и др. (на последнем месте любой символ).
?	после символа или группы символов указывает, что символ или группа повторяется 0 или 1 раз	/ab?c/	"ac" и "abc"
()	"запоминающие скобки" — объединяют символы в группу, и результат поиска запоминается в RegExpr. Используются и для иных целей (см. след. метасимвол)	/(ab)?c/	"c" и "abc"
	логическое "или" между частичными шаблонами в общих скобках.	/(a b)1/ /(a b c)1/	"ac" и "bc" "a1", "b1" и "c1"
*	после символа или группы символов указывает, что символ или группа повторяется 0,	/ab*c/	"ac", "abc", "abbc", "abbbc", "abbbbc" и т.д.

	1 и любое большее число раз		
+	после символа или группы символов указывает, что символ или группа повторяется 1, 2 и любое большее число раз	/ab+c/	"abc", "abbc", "abbbc", "abbbbc" и т.д.
{n}	после символа или группы символов (n — число) — число повторений.	/ab{3}c/	"abbbc"
[]	ограничивают описание набора значений для единичного символа. Символы набора перечисляются, задаются в форме диапазона (начало и конец через дефис) или то и другое вместе.	/ab[c12]/ /a[Aa-z2-7]/	"abc", "ab1" и "ab2" все фрагменты, у которых в начале "a", а потом: или символ "A", или любой символ от "a" до "z", или цифра от 2 до 7.
\d	любая цифра	/a\dс/	"a0с", "a1с", "a2с" и т.д., но не "abc".
\D	любой символ, кроме цифры	/a\Dс/	"abc", "aac", "a=c" и т.д., но не "a1с".
\s	любой пробельный символ, включая пробел, табуляция, перевод строки и др.	/\sb\d/	" ba" в строке "foobi bar"
^	начало строки (вне квадратных скобок)	/^abc/	"abc" в строке " <u>abc</u> d eg f", но не в строке "d <u>abc</u> eg"
[^]	инвертирование (отрицание) указанного набора значений единичного символа	/[^abc]/	любой символ, кроме "a", "b" и "с".
\$	конец строки	/abc\$/	"abc" в строке "eg fd <u>abc</u> ", но не в строке "g

			<u>abcd</u> egf"
--	--	--	------------------

4. Флаги регулярных выражений

Флаги регулярных выражений дописываются к шаблону справа.

Всего возможных флагов три:

g — флаг глобального поиска (по умолчанию поиск в строке прерывается, если найден первый подходящий фрагмент). Флаг **g** позволяет производить поиск по всей строке и получать все подходящие фрагменты;

i — флаг поиска без учета регистра (по умолчанию регистр учитывается);

m — флаг многострочного поиска. При указании этого флага строка, содержащая символы перевода строки, рассматривается как несколько строк, и поиск ведется в каждой такой строке (по умолчанию символы перевода строки рассматриваются как обычные символы).

Флаги можно не указывать, и поиск будет проводиться с параметрами по умолчанию. Записывать флаги можно в любой последовательности. Например, следующие регулярные выражения, в которых есть все три флага, абсолютно идентичны:

/abc\d+/gmi, /abc\d+/img, /abc\d+/mig

5. Объекты типа RegExp

Работа с регулярными выражениями на языке JavaScript поддерживается встроенными объектами **RegExp** и **String**.

Объект RegExp на языке JavaScript называют глобальным.

Все регулярные выражения и переменные, которые принимают регулярные выражения, являются копиями объекта **RegExp** и называются **объектами типа RegExp**.

Вот, например, описание переменной **reg** как объекта типа **RegExp**:

```
var reg = new RegExp();
```

При описании переменной типа **RegExp** ей одновременно можно задавать значение. Например:

```
var reg = new RegExp("abc\\d", "gmi");
```

Значения шаблона и флагов задаются как два аргумента типа строка.

В строке шаблона специальный символ **"\"** (только он) обязательно предваряется еще одним таким же.

Если в регулярном выражении флагов нет, то при описании значения достаточно одного аргумента.

```
var reg = new RegExp("abc\\d");
```

Объекты типа **RegExp** в программе на языке JavaScript также можно создать с помощью оператора присваивания (в форме литерала, без кавычек). Например,

```
var reg1 = /abc\d+/gmi;  
var reg2 = /abcdg/;
```

6. Методы объекта типа *RegExp*

Пусть некоторое регулярное выражение задано как значение объекта типа **RegExp**. Тогда для этого объекта известны следующие методы:

test() — проверяет наличие подходящих фрагментов в строке, которая служит аргументом метода. Если поиск успешен, метод возвращает **true**, если неудачен — **false**;

exec(s) — проверяет наличие подходящих фрагментов в строке, которая служит аргументом метода. Если регулярное выражение не содержит круглых скобок, то метод возвращает первый подходящий фрагмент, если поиск неудачен, метод возвращает **null**.

Пример 1.

С помощью методов **test()** и **exec()** проведем в произвольных строках поиск фрагмента из трех символов, первые два из которых **vc**.

В нашем случае шаблон имеет вид **/vc./**. Составляем программу.

Выводим на чистую вкладку редактора обычную HTML-конструкцию. Записываем заголовок «Регулярные выражения 1». Сохраняем файл под именем **reg1.html**.

В раздел **<script></script>** записываем сначала шаблон, как значение объекта типа **RegExp**

```
var reg=/vc./;
```

Потом для поиска задаем две строки

```
var s1="asdfghjkmnbxz";  
var s2="asdfgvcxhjkmnvcxbz";
```

Первая строка **s1** не содержит фрагмента **"vc."**, а вторая **s2** содержит дважды.

Теперь вычисляем результаты проверки строк методом **test()** и сразу выводим эти результаты в окно браузера

```
document.write(reg.test(s1)+'<br>'+reg.test(s2));
```

Сохраняем файл и запускаем его на исполнение. Успешен поиск в **s2**.

Теперь дописываем команду с методом **exec()**

```
document.write('<br>'+reg.exec(s1)+'<br>'+reg.exec(s2));
```

Сохраняем файл и запускаем его на исполнение. Опять успешен поиск в **s2**, и выдается первый подходящий фрагмент.\

Если регулярное выражение содержит хоть одну группу символов (в круглых скобках), то метод **exec()** возвращает массив фрагментов. В этом массиве элемент с индексом 0 — первый фрагмент, подходящий всему шаблону, следующий элемент (с индексом 1) — фрагмент элемента с индексом 0, который подходит под шаблон-группу (выделенную круглыми скобками). И т.д.

Пример 2. Проверим действие метода **exec()** в случае сложного шаблона.

Воспользуемся файлом **reg1.html**. Заменяем заголовок на «Регулярные выражения 2» и сохраняем файл под именем **reg2.html**.

В шаблон добавим круглые скобки

```
var reg=/v(c.)//;
```

Сохраняем файл и запускаем его на исполнение. Опять успешен поиск в **s2**, и в результате выдаются два фрагмента через запятую, как значения элементов массива.

Добавим еще пару скобок в регулярное выражение

```
var reg=/v((c).)/;
```

Сохраняем файл и запускаем его на исполнение. Опять успешен поиск в **s2**, и выдаются уже три фрагмента. В скобки, например, можно заключить еще и символ **"v"**.

7. Свойства глобального объекта *RegExp*

Не совсем обычно то, что глобальный объект **RegExp**, с помощью которого образуются объекты этого типа, имеет свои свойства. Эти свойства хранят данные последнего поиска одним из методов **test()** или **exec()**. Так:

RegExp.input — содержит всю строку, в которой выполнялся поиск;

RegExp.index — содержит индекс позиции, начиная с которой расположен подходящий фрагмент;

RegExp.leftContext — содержит текст из строки поиска от начала строки до начала подходящего фрагмента;

RegExp.rightContext — содержит текст из строки поиска от конца подходящего фрагмента до конца строки.

Пример 3.

Выведем на экран значения свойств глобального объекта **RegExp** после применения метода **exec()** к нашему объекту **reg** в примере 1.

Воспользуемся файлом **reg1.html**. Заменяем заголовок на «Регулярные выражения 3» и сохраняем файл под именем **reg3.html**.

Дописываем в текст команды

```
document.write("<br>" + RegExp.input + "<br>" + RegExp.index);  
document.write("<br>" + RegExp.leftContext);  
document.write("<br>" + RegExp.rightContext);
```

Сохраняем файл и запускаем его на исполнение. Все перечисленные ранее значения свойств действительно выводятся.

8. Методы объекта *String* для работы с регулярными выражениями

String — это объект «строка». Все строки автоматически являются объектами типа **String**. Для этих объектов мы уже неоднократно пользовались, например, свойством **length** (длина). Для этих объектов мы также использовали метод выделения фрагментов **substr()**, методы упорядочения **sort()** и **reverse()** (см. часть «Ядро JavaScript 3»).

Объект имеет также методы, которые работают с регулярными выражениями. Среди них:

search() — аналог метода **test()**, где аргументом служит регулярное выражение;

match(r) — аналог метода **exec()**, где аргументом служит регулярное выражение. Заметим, что если в регулярном выражении установлен флаг **g**, метод ищет в строке все подходящие фрагменты и возвращает массив таких фрагментов.

Это действительно почти полные аналоги упомянутых методов для объектов типа **RegExp**. В тех методах объектом служит *регулярное выражение*, а аргументом — *строка*. А в этих методах объектом служит *строка*, а аргументом — *регулярное выражение*.

Пример 4.

Зададим строку, содержащую числа, и попробуем методом **match()** собрать эти числа в массив.

Шаблон, который находит только числа, имеет вид **/\d+/**. Чтобы создавался массив, к нему нужно дописать флаг **g**.

Выводим на чистую вкладку обычную HTML-конструкцию. Задаем заголовок «Регулярное выражение 4». Сохраняем файл под именем **reg4.html**.

В раздел **<script></script>** записываем шаблон

```
var reg2 = /\d+/g;
```

Строку

```
var s1 = "ID = 123 №23 abc345def";
```

Затем выводим исходную строку и результат применения метода `match()`

```
document.writeln(s1 + "<br>" + s1.match(reg2));
```

Сохраняем файл и запускаем его на исполнение. На экран выводятся найденные элементы числового массива.

Для объекта типа **String** есть еще один особенный метод:

replace(r,ss), где **r** — регулярное выражение, а **ss** — некоторая строка. Метод производит замену подходящих по **r** фрагментов строки на значение строки **ss**.

Пример 5.

В условиях примера 4 заменим теперь все числа, входящие в строку **s1** на «000» (три нуля).

Воспользуемся файлом **reg4.html**. Заменим заголовок на «Регулярные выражения 5» и сохраняем файл под именем **reg5.html**.

В раздел `<script> </script>` дописываем

```
var s2="000";  
document.writeln("<BR>" + s1.replace(reg2,s2));
```

Сохраняем файл и запускаем его на исполнение.

Проверим также, меняет или не меняет исходную строку применение метода. Напомним, что методы **sort()** и **reverse()** исходную строку меняли (см. часть «А2 Ядро JavaScript»).

Для проверки дописываем команду вывода исходной строки после применения метода:

```
document.writeln("<BR>" + s1);
```

Сохраняем файл и запускаем его на исполнение.

Замечаем, что применение метода **replace()** исходную строку не меняет.

9. Применение регулярных выражений

Регулярные выражения используются для поиска строковых фрагментов в длинных строках и для проверки данных, вводимых в формы.

Большинство случаев применения регулярных выражений в скриптах JavaScript, конечно, связано с проверкой корректности данных, вводимых пользователем в поля форм. Такая проверка позволяет избежать отправки заведомо некорректных данных обработчикам этих форм и известить пользователей о совершенных ими ошибках, повышая тем самым эксплуатационные характеристики веб-сервисов. Рассмотрим несколько примеров использования.

10. Поиск записи cookie по ее имени.

Поиск записи cookie по ее имени в строке **document.cookie**. можно организовать с помощью регулярного выражения (шаблона).

Составим такое регулярное выражение. Пусть в строке **document.cookie** ищется содержание записи cookie с именем, которое является значением переменной **name**. Будем считать, что имя cookie включает латинские буквы и цифры.

В строке **document.cookie** записи разделяются парой символов **"; "** (точка с запятой и "пробел"). Потому имя считываемой записи надо искать в начале всей строки или после окончания каждой отдельной записи. Для поиска таких позиций в строке подходит шаблон

(^|;)

который означает — найти начало строки записей ("**^**") или ("**|**") пару символов

("; "). Круглые скобки ограничивают действие логического "или".

Далее в регулярное выражение должно быть записано имя записи (у нас это значение переменной **name**).

В записи cookie имя и содержание записи разделяет знак равенства, поэтому после нахождения имени в строке надо выделить все символы, начиная со знака равенства **"="** и кончая следующей точкой с запятой **"; "**.

Для этого подходит шаблон

=[^;]*

который означает знак равенства **"="**, после которого идут любые символы, кроме точки с запятой ("**[^;]**"), и в любом количестве (символ **"*"** за квадратными скобками). Значит, все символы до точки с запятой.

Нам надо извлекать данные записи (после знака равенства), поэтому искомые символы после знака **"="** заключим в круглые скобки. Так мы создали группу символов, готовясь к поиску методом **exec()** или **match()**:

=(^[^;]*)

Получили две заготовки для регулярного выражения:

(^|;) и **=(^[^;]*)**

Между ними надо записать значение переменной **name** и образовать единый шаблон.

Это можно сделать только одним способом. Нужно склеить все три записи как строки (с помощью знака «+» склейки):

"(^|;)" + name + "=(^[^;]*)"

а потом преобразовать полученную строку в регулярное выражение, с помощью команды

```
var reg= new RegExp("(^|; )" + name + "=(^[^;]*)");
```

Регулярное выражение построено.

Пример 6.

В разделе 15 "Работа с cookie" части «Работа с объектами DOM 3» для чтения записи cookie по ее имени была введена функция **readCookie()**.

Построим другой вариант этой функции, в котором поиск организован на основе регулярного выражения.

Воспользуемся файлом **pos41.html**, изменяем в нем заголовок на «Регулярные выражения 6» и сохраняем под именем **reg6.html**.

Изменяем функцию **readCookie()**. Очищаем все строки, кроме последней. Первой строкой записываем команду объявления полученного регулярного выражения:

```
var reg= new RegExp("(?:^|; )" + name + "=(^[^;]*)");
```

Поиск в строке записей **document.cookie** организуем методом **.exec()**. Поэтому второй командой записываем

```
var exec = reg.exec(document.cookie);
```

Шаблон поиска имеет структуру

```
/(^|; )имя_записи=(^[^;]*)/
```

Значит, результатом будет массив **exec** с тремя элементами:

- * с индексом 0: первый фрагмент строки как совпадение всего шаблона
- * с индексом 1: фрагмент элемента с индексом 0 как совпадение шаблона первой группы (в первых скобках);
- * с индексом 2: фрагмент элемента с индексом 0 как совпадение шаблона второй группы (во вторых скобках, которые для этого и введены).

Нам нужен третий, значит, выводим элемент **exec[2]**, который надо предварительно раскодировать функцией **unescape()**.

Учитывая это, последнюю (не удаленную) команду меняем на

```
return exec ? unescape(exec[2]) : undefined;
```

Она означает, если запись cookie с таким именем есть, выводится декодированное значение результата поиска **unescape(exec[2])**. Если записи cookie с таким именем нет, выводится значение **undefined**.

Заметим, что функция **readCookie()** существенно сократилась в объеме. Сохраняем файл и запускаем его на исполнение.

(!!! Пример 6 после выполнения показать преподавателю!!! 32)

11. Контроль корректности ввода даты.

Часто в формах регистрации учетных записей веб-сервисов требуется указать даты (своего рождения, выездов за границу и т.п.). Обычно для ввода даты предусматривается несколько полей (отдельные поля для ввода года, месяца, числа месяца). Однако для этой цели часто используют одно текстовое

поле, предложив пользователю ввести дату в формате **гггг-мм-дд** с разделителем "-", где **дд** — число месяца (две цифры), **мм** — номер месяца (две цифры), **гггг** — год (четыре цифры). Именно в таком виде, например, требуется вводить даты в интернет-анкетах, направляемых в посольства стран Шенгенского соглашения.

Пример 7.

Создадим веб-страницу с формой, которая содержит текстовое поле и две кнопки "Очистить" и "Проверить", для проверки корректности ввода даты в формате

гггг-мм-дд. Допустимо вводить число месяца от 01 до 31 (две цифры), номер месяца от 01 до 12 (две цифры) и год в диапазоне от 1900 (четыре цифры). В ответ на нажатие кнопки "Проверить" скрипт должен проверить введенные данные и в диалоговом окне дать заключение "Верно" или "Неверно".

Составим регулярное выражение для проверки корректности вводимых данных.

Сначала составим шаблон, который будет проверять допустимые значения года. Первыми двумя символами строки для года могут только 19 или 20.

Первый случай (19):

1) Если первые две цифры года 19, то следующие две любые от 0 до 9, что записывается **19[0-9][0-9]**.

Напомним, что квадратные скобки имеют смысл перечисления возможных элементов.

Второй случай (20) разделим на два:

2) Если первые три цифры года 200, то следующие от 0 до 9, что записывается **200[0-9]**.

3) Если первые две цифры года 201, то следующие 0 до 5, что записывается **201[0-5]**.

Все три случая объединяются символами «|» («или») и общими круглыми скобками в группу

(19[0-9][0-9]|200[0-9]|201[0-5])

Теперь составим шаблон для данных номера месяца.

Номер месяца должен находиться в диапазоне от 01 до 12.

В случае, если первая цифра 0, то вторая может изменяться от 1 до 9 — это шаблон **0[1-9]**. Если же первая цифра 1, то вторая изменяется от 0 до 2 — это шаблон **1[0-2]**. Объединяем символом «|» («или»), скобками и получаем группу символов:

(0[1-9]|1[0-2])

Осталось составить шаблон, который будет проверять допустимые значения числа месяца.

Для простоты не будем следить за тем, что не любые числа месяца могут соответствовать конкретному месяцу и году (например, невозможно 30 февраля). Тогда для числа месяца имеются три возможности:

1) 0 и цифра от 1 до 9, что записывается **0[1-9]**.

2) Цифра от 1 до 2 и цифра от 0 до 9, что записывается **[1-2][0-9]**.

3) Цифра 3 и цифра от 0 до 1, что записывается **3[0-1]**.

Все три возможности объединяются символами «|» («или») и общими круглыми скобками

(0[1-9]|[1-2][0-9]|3[0-1])

Объединяем в один шаблон три группы, записывая между ними разделительный символ "-". Результат заключаем в дробные знаки. Заметим, что шаблон **[0-9]** равносильен метасимволу **\d**, но последний короче.

Получаем регулярное выражение

/(19\d\d|200\d|201[0-4])-(0[1-9]|[1-2][0-9]|3[0-1])/

Проверку введенной строки по шаблону удобно производить с помощью метода **.test()** объекта типа **RegExp** (объект — регулярное выражение, аргумент — строка).

Файл примера с именем **reg7.html** создать самостоятельно.

(!!! Пример 7 после выполнения показать преподавателю!!! 33)

11. Контроль корректности ввода адреса электронной почты.

Пример 8.

Создадим веб-страницу с формой, которая содержит текстовое поле и две кнопки "Очистить" и "Проверить", для проверки корректности ввода адреса электронной почты.

Будем считать, что в записи адреса электронной почты русские буквы не используются (только латинские). Примеры адресов:

max.semenov@mail.ru

_ira_motuz-glushko@bspu.unibel.by

volich12@go-mail.by

Составим регулярное выражение для проверки корректности вводимых данных.

Заметим, что в левой части адреса до знака "@" встречаются наборы букв и цифр, разделенные знаками ".", "_" и "-".

В правой части тоже группы и цифр, разделенные знаками "." и "-".

Набор латинских букв и цифр задает шаблон

[a-zA-Z0-9]+

Шаблон для разделяющих символов в левой части адреса имеет вид

[-_\.]

Тогда левую часть можно проверить шаблоном

[a-zA-Z0-9]+([-_\.][a-zA-Z0-9]+)+

Последний "плюс" и «плюс» перед круглой скобкой означают, что указанных до них наборов может быть несколько.

Добавим слева знак "^" начала записи и шаблон "_?", так как имя пользователя *иногда* начинается с символа подчеркивания. Получаем шаблон левой части адреса

^_?[a-zA-Z0-9]+([-_\.][a-zA-Z0-9]+)+

Правая часть адреса содержит доменное имя, которое включает наборы букв и цифр, разделенные знаками "-" или ".", что можно записать в виде

[a-zA-Z0-9]+([-_\.][a-zA-Z0-9]+)+\$

Последним добавлен знак "\$" окончания строки.

Объединяем частичные выражения, вставляя символ "@", и получаем длинное регулярное выражение

/^_?[a-zA-Z0-9]+([-_\.][a-zA-Z0-9]+)+@([a-zA-Z0-9]([-_\.])+[a-zA-Z0-9]+)+\$/

Шаблон проверяет электронные адреса достаточно корректно.

Файл примера с именем **reg8.html** создать самостоятельно.

(!!! Пример 8 после выполнения показать преподавателю!!! 34)

Задания.

На базе примеров 7, 8 создать веб-страницы для проверки корректности следующих данных:

1) Номера паспорта гражданина РБ.

Номер паспорта имеет в составе две большие латинские буквы и семиразрядное десятичное число.

2) Государственного автомобильного номера РБ.

Пример номера: 0235КН-5. Входят 4 цифры, две белорусские прописные буквы, которые по написанию совпадают с латинскими. После дефиса цифра от 0 до 7 (номер региона).

3) Идентификационного номера (из паспорта гражданина РБ).

Первая цифра 3 или 4, далее 6 цифр даты рождения (год сокращен до 2-х цифр), буква, 3 цифры, 2 буквы, цифра. Входят русские прописные буквы, которые по написанию совпадают с латинскими.

4)* Даты рождения с учетом того, что не любые числа месяца могут соответствовать конкретному месяцу и году. Например, в любом году невозможно 31 июня, а в невисокосном году нет 29 февраля.